

Audio Processing in NLP

Naeemullah Khan

naeemullah.khan@kaust.edu.sa

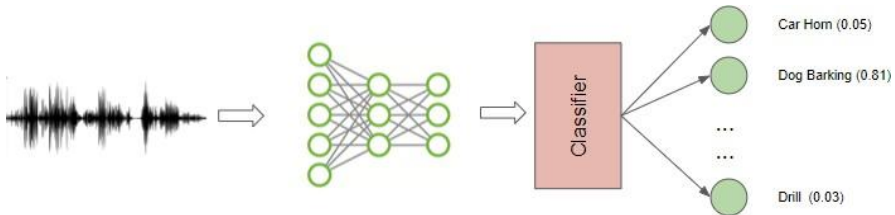


جامعة الملك عبد الله
للعلوم والتقنية

King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

Aug 19, 2026



1. [Motivation](#)
2. [Learning Outcomes](#)
3. [Fundamentals of Sound](#)
4. [Pre-processing Pipeline](#)
5. [STFT & Spectrograms](#)
6. [ASR Evolution](#)
7. [Connectionist Temporal Classification \(CTC\)](#)
8. [Seq2Seq ASR with Attention](#)
9. [RNN-Transducer \(RNN-T\)](#)
10. [Transformer & Conformer in ASR](#)

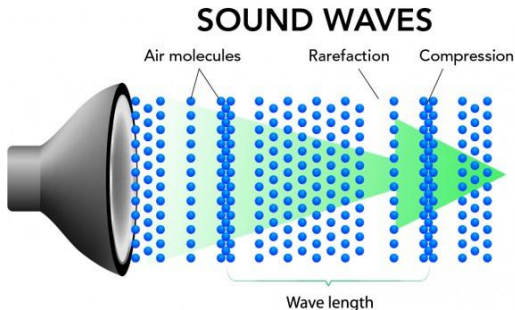
14. [Self-Supervised Learning: Wav2Vec 2.0](#)
15. [Fine-tuning for ASR with Wav2Vec 2.0](#)
16. [Conformer-CTC: Non-autoregressive ASR](#)
17. [Advanced Tasks: Diarization & Emotion Recognition](#)
18. [Text-to-Speech \(TTS\) & Vocoder](#)
19. [Challenges & Current Research](#)
20. [Summary](#)
21. [References](#)

- ▶ Audio data is abundant and diverse, encompassing speech, music, and environmental sounds.
- ▶ Traditional NLP methods focus on text, but audio data requires specialized techniques for processing.
- ▶ The rise of voice assistants and audio-based applications highlights the need for effective audio processing in NLP.
- ▶ Audio signals contain rich information that can enhance understanding and interaction in various applications.

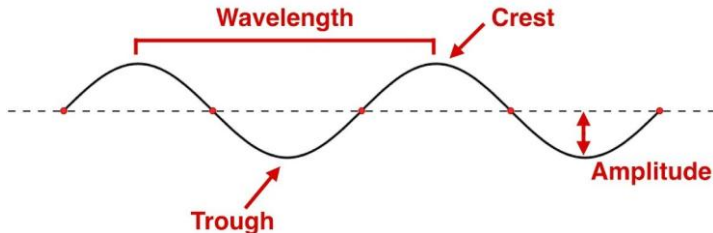
By the end of this session, you will be able to:

- ▶ Understand the fundamentals of audio processing in NLP.
- ▶ Learn how to extract features from raw audio data.
- ▶ Explore various architectures for Automatic Speech Recognition (ASR).
- ▶ Gain insights into self-supervised learning models for audio.
- ▶ Implement Text-to-Speech (TTS) systems using modern techniques.
- ▶ Address advanced audio tasks such as speaker diarization and emotion recognition.
- ▶ Recognize the challenges and future directions in audio processing for NLP.

Fundamentals of Sound



- ▶ Sound is a type of energy produced by vibrating objects.
- ▶ It travels through air (or other medium) as waves.
- ▶ Sound waves are longitudinal waves, meaning the particle displacement is parallel to the direction of wave propagation.



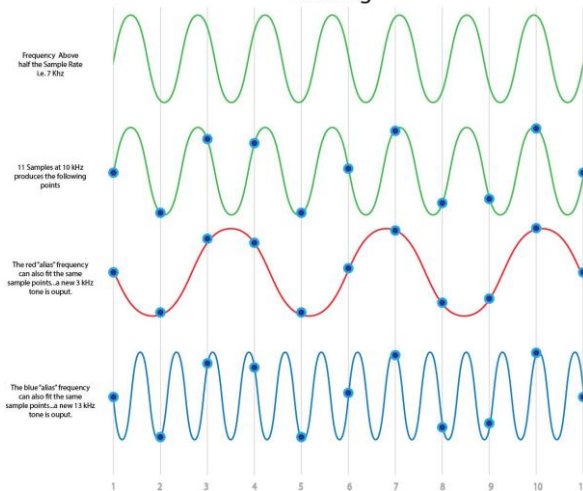
- ▶ **Waveform:** The shape of the sound wave, representing how air pressure changes over time.
- ▶ **Frequency:** Number of cycles (oscillations) per second, measured in Hertz (Hz). Determines the pitch of the sound.
- ▶ **Amplitude:** The height of the wave, representing the loudness or intensity of the sound.

- ▶ **Sampling Theorem (Nyquist):** To accurately digitize a sound, the sampling rate (f_s) must be greater than twice the highest frequency present in the signal (f_{max}):

$$f_s > 2f_{max}$$

- ▶ **Aliasing:** If the sampling rate is too low, higher frequencies are misrepresented as lower frequencies, causing distortion.

Aliasing



Pre-processing Pipeline

1. Pre-emphasis Filter

- ▶ Enhances high-frequency components of the audio signal.
- ▶ Equation:

$$y[n] = x[n] - \alpha x[n - 1]$$

where $x[n]$ is the input signal, $y[n]$ is the output, and α is typically between 0.95 and 0.97.

- ▶ Helps balance the frequency spectrum and improves feature extraction.

2. Framing

- ▶ Audio signals are divided into short frames (e.g., 25 ms) to capture local temporal features.
- ▶ Each frame overlaps with the next (commonly 10 ms overlap).

3. Windowing

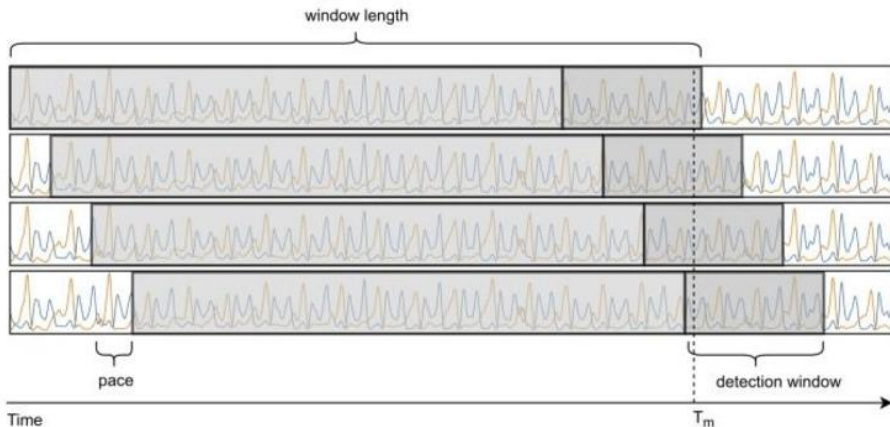
- ▶ Each frame is multiplied by a window function (commonly Hamming window) to reduce spectral leakage.
- ▶ Hamming window equation:

$$w[n] = 0.54 - 0.46 \cos\left(\frac{2\pi n}{N-1}\right)$$

where N is the frame length.

Diagram: Waveform → Framed Windows

Pre-processing Pipeline Details (cont.)



Waveform segmented into overlapping frames and windowed (source: ResearchGate)

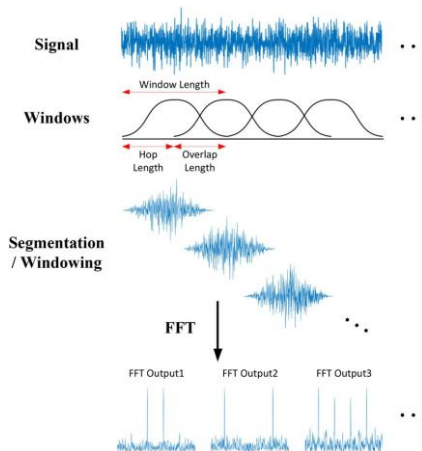
Summary of Pre-processing Steps

1. **Pre-emphasis:** Boosts high frequencies.
2. **Framing:** Splits signal into short, overlapping segments.
3. **Windowing:** Applies Hamming window to each frame.

STFT & Spectrograms

- ▶ STFT is a method to analyze the frequency content of a signal over time.
- ▶ It involves applying the Fourier Transform to short overlapping segments (frames) of the signal.
- ▶ The STFT provides a time-frequency representation of the signal.

Short-Time Fourier Transform (STFT) (cont.)



STFT process: Signal divided into frames, each transformed into frequency domain.

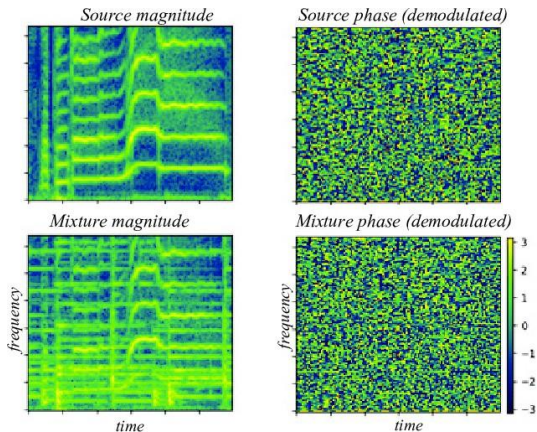
- The STFT is computed as:

$$STFT(x[n]) = \sum_{m=-\infty}^{\infty} x[m]w[n-m]e^{-j2\pi fm}$$

where $w[n-m]$ is a window function applied to each frame.

- ▶ The result is a complex-valued matrix, where each column represents the frequency content of a frame.
- ▶ The magnitude of the STFT gives the amplitude of each frequency at each time step.
- ▶ The phase of the STFT provides information about the timing of the frequency components.

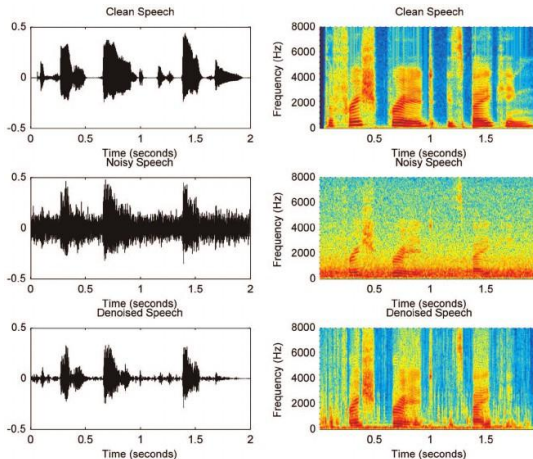
Short-Time Fourier Transform (STFT) (cont.)



Magnitude and phase of STFT: Magnitude shows amplitude, phase shows timing of frequencies.

- ▶ A spectrogram is a visual representation of the STFT.
- ▶ It displays time on the x-axis, frequency on the y-axis, and color intensity represents amplitude.
- ▶ Spectrograms are widely used in audio analysis, speech recognition, and music processing.

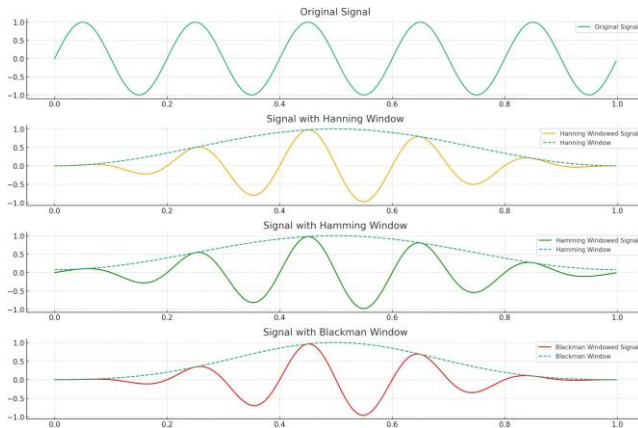
Spectrograms (cont.)



Example of a spectrogram: Time vs Frequency representation of an audio signal.

- ▶ Spectrograms can be computed using various window functions (e.g., Hamming, Hanning) and different frame sizes.
- ▶ They provide insights into how the frequency content of a signal changes over time.

Spectrograms (cont.)



Spectrogram variations: Different window functions and frame sizes affect the appearance of the spectrogram.

History of ASR

1952 - Early Beginnings

Bell Laboratories developed the Audrey system, which could recognize digits spoken by a single speaker.

1971 - Rule-Based Systems

The Harpy system, developed by Carnegie Mellon University, could understand about 1,011 words using a more sophisticated pattern recognition approach.

2011 - Deep Learning Revolution

Apple introduced Siri, a voice-activated assistant that utilized ASR and natural language processing (NLP).

2020 - Advanced AI Integration

OpenAI's GPT-3 showed advanced capabilities in understanding and generating human-like text, enhancing ASR applications in transcription and interactive systems.

1962 - First Generation Systems

IBM created the Shoebox machine, capable of recognizing 16 spoken words and digits.

1980 - Statistical Models

Introduction of the Hidden Markov Model (HMM) to ASR, which became the standard for statistical speech recognition.

2016 - Generative Models

Google's WaveNet demonstrated the potential of deep generative models for high-fidelity speech synthesis.

2022 - End-to-End (E2E) Modeling

Development of enterprise-level neural ASR models, multilingual models, and end-to-end modeling. There was also a push towards integrating speech enhancement and diarization to improve performance in challenging acoustic environments.

- Speech recognition Evaluation metrics
- Standard speech recognition pipeline
- (a bit) mathematical formulation of speech recognition

- Sentence error rate
 - An entire sentence (utterance) is correct or not (100% error in the case below)

Reference

I want to go to the CMU campus

Recognition result

I want to go to the gym you can

- Too strict, and needs to consider some local correctness

- Word error rate (WER)
 - Using edit distance word-by-word:

Reference

I want to go to the CMU campus

Recognition result

I want to go to the gym you can

- # insertion errors = 1, # substitution errors = 2, # of deletion errors = 0 $\Rightarrow$ Edit distance = 3
- Word error rate (%): $\text{Edit distance (=3)} / \text{\# reference words (=8)} * 100 = 37.5\%$
- How to compute WERs for languages that do not have word boundaries?
 - Chunking or using character error rate

- Character error rate (CER)
 - Using edit distance character-by-character:

Reference)

CMU大学のキャンパスに行きたいで

す Recognition result)

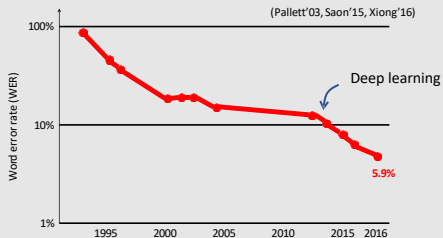
CMU洋楽のキャンパスに行きたいです

- # insertions = 0, # substitutions = 2, # deletions = 0
 → Edit distance = 2
- Character error rate (%): $\text{Edit distance} (=2) / \text{\# reference words} (=18) * 100 = 11.1\%$

- Other metrics
 - Phoneme error rate (need a pronunciation dictionary)
 - Frame error rate (need an alignment)
 - Real time factor and latency (for a speech interface product)
 - Downstream evaluation metrics (e.g., BLUE if we combine it with machine translation)
- NIST Speech Recognition Scoring Toolkit (SCTK)
 - <ftp://jaguar.ncsl.nist.gov/pub/sctk-2.4.10-20151007-1312Z.tar.bz2>
 - Standardize these metrics

Speech recognition research is easy (?)

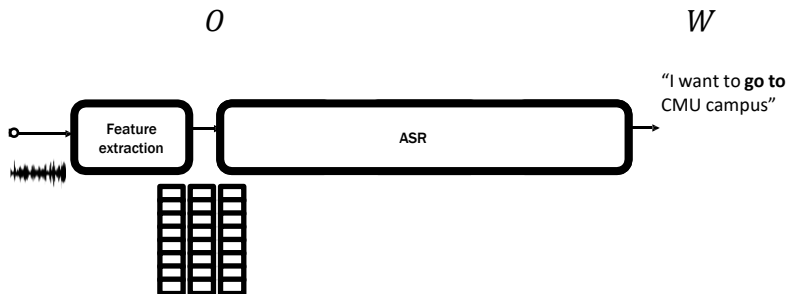
- We have WER or CER
- Objective, easy to obtain, very application-specific, single objective
- This can show the clear progress of technologies
- This would be one reason that the effectiveness of deep learning was first shown in speech



- Speech recognition demo & Evaluation metrics
- **Standard speech recognition pipeline**
- (a bit) mathematical formulation of speech recognition

Automatic speech recognition

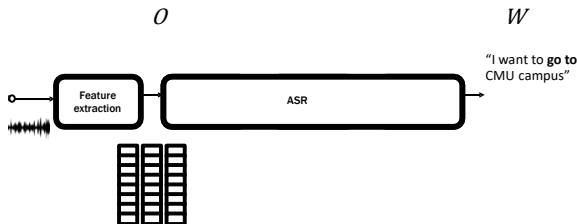




- Instead of starting from the waveform, we will often start from **speech features** (MFCC, etc.) through the **feature extraction** module
- Let's think of the conversion from speech feature O to text W

- **MAP decision theory**: Estimate the most probable word sequence $\hat{W}$ among all possible word sequences W (omit the domain sometimes)

$$\hat{W} = \operatorname{argmax}_{W \in \mathcal{W}} p(W|O)$$



Probabilistic rules

- **Product rule**

$$p(x|y)p(y) = p(x, y)$$

- **Sum rule**

$$p(y) = \sum_x p(x, y)$$

- **Conditional independence assumption**

$$p(x|y, z) = p(x|z) \quad p(x, y|z) = p(x|z)p(y|z)$$

How to obtain the posterior $p(W|O)$

- Noisy channel model
 - Regarding O as a probabilistic variable (noisy observation)
 - Use the product rule

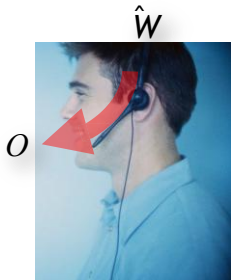
$$\begin{aligned}\operatorname{argmax}_W p(W|O) &= \operatorname{argmax}_W \frac{p(O|W)p(W)}{p(O)} \\ &= \operatorname{argmax}_W \underbrace{p(O|W)}_{\text{Likelihood}} \underbrace{p(W)}_{\text{Prior}}\end{aligned}$$



How to obtain the posterior $p(W|O)$

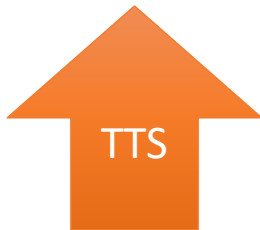
- Noisy channel model

$$\begin{aligned}\operatorname{argmax}_W p(W|O) &= \operatorname{argmax}_W \frac{p(O|W)p(W)}{p(O)} \\ &= \operatorname{argmax}_W p(O|W)p(W)\end{aligned}$$



- Solving generating process of noisy observation!!
- Still difficult to deal with them....

Speech O :



Text W : I want to go to the CMU campus

Speech O :



Phoneme L : AY W AA N T T UW G OW T UW DH AH S IY EH M Y UW K AE M P AH
S



Text W : I want to go to the CMU campus

- Further factorize the model with **phoneme**
 - Let $L = (l_i \in \{ /AA/, /AE/, \dots \} | i = 1, \dots, J)$ be a phoneme sequence

$$\arg \max_W p(W|O)$$

- Further factorize the model with **phoneme**
 - Let $L = (l_i \in \{/AA/, /AE/, \dots\} | i = 1, \dots, J)$ be a phoneme sequence

$$\arg \max_W p(W|O) = \arg \max_W \sum_L p(W, L|O) \quad \text{Sum rule}$$

- Further factorize the model with **phoneme**
 - Let $L = (l_i \in \{ /AA/, /AE/, \dots \} | i = 1, \dots, J)$ be a phoneme sequence

$$\arg \max_W p(W|O) = \arg \max_W \sum_L p(W, L|O) \quad \text{Sum rule}$$

$$= \arg \max_W \sum_L \frac{p(O|W, L)p(L|W)p(W)}{p(O)} \quad \text{Product rule}$$

- Further factorize the model with **phoneme**
 - Let $L = (l_i \in \{/AA/, /AE/, \dots\} | i = 1, \dots, J)$ be a phoneme sequence

$$\arg \max_W p(W|O) = \arg \max_W \sum_L p(W, L|O)$$

Sum rule

$$= \arg \max_W \sum_L \frac{p(O|W, L)p(L|W)p(W)}{p(O)}$$

Product rule

$$= \arg \max_W \sum_L p(O|W, L)p(L|W)p(W)$$

Ignore $p(O)$ as it does not depend on W

- Further factorize the model with **phoneme**
 - Let $L = (l_i \in \{/AA/, /AE/, \dots\} | i = 1, \dots, J)$ be a phoneme sequence

$$\arg \max_W p(W|O) = \arg \max_W \sum_L p(W, L|O)$$

Sum rule

$$= \arg \max_W \sum_L \frac{p(O|W, L)p(L|W)p(W)}{p(O)}$$

Product rule

$$= \arg \max_W \sum_L p(O|W, L)p(L|W)p(W)$$

Ignore $p(O)$ as it does not depend on W

$$= \arg \max_W \sum_L p(O|L)p(L|W)p(W)$$

Conditional independence assumption

$$\begin{aligned}\arg \max_W p(W|O) &= \arg \max_W p(O|W)p(W) \\ &\approx \arg \max_W \sum_L p(O|L)p(L|W)p(W)\end{aligned}$$

- **Speech recognition**

- $p(O|L)$: Acoustic model (Hidden Markov model)
- $p(L|W)$: Lexicon
- $p(W)$: Language model (n-gram)

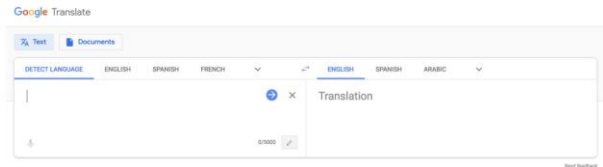
W : Target language text

Y : Source language text

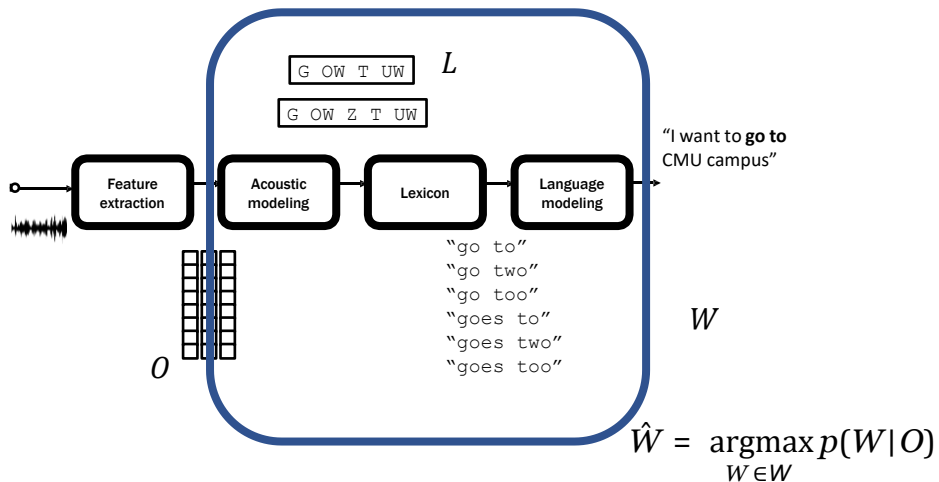
$$\arg \max_W p(W|Y) = \arg \max_W p(Y|W)p(W)$$

- **Machine translation**

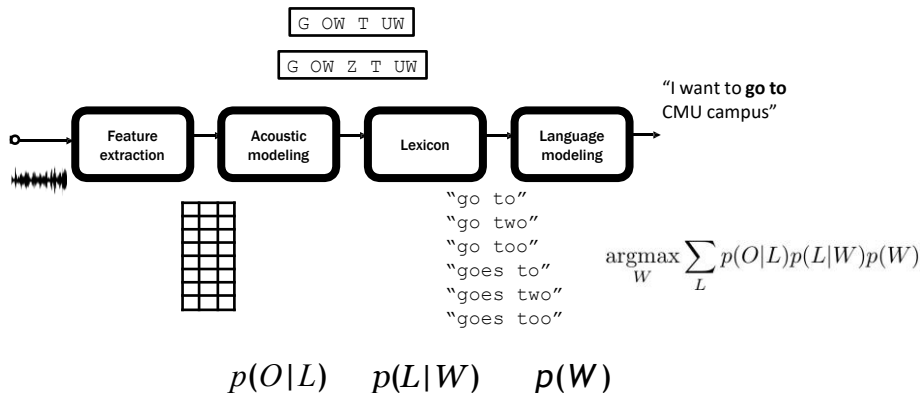
- $p(Y|W)$: Translation model
- $p(W)$: Language model



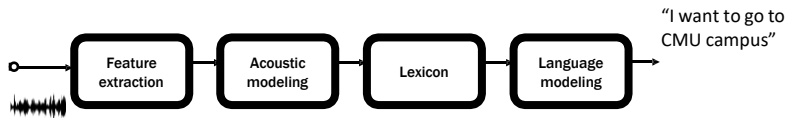
Speech recognition pipeline



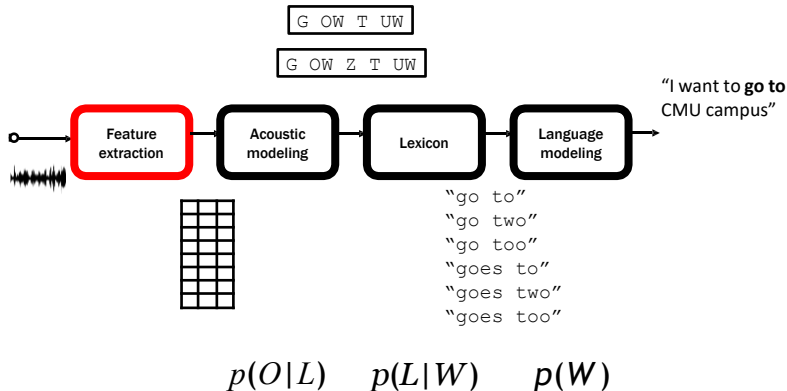
Speech recognition pipeline

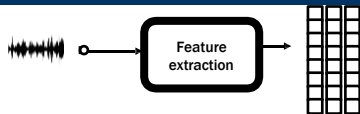


- Speech recognition demo & Evaluation metrics
- (a bit) mathematical formulation of speech recognition
- **Standard speech recognition pipeline**



Speech recognition pipeline

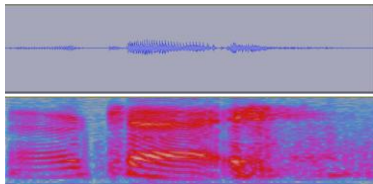




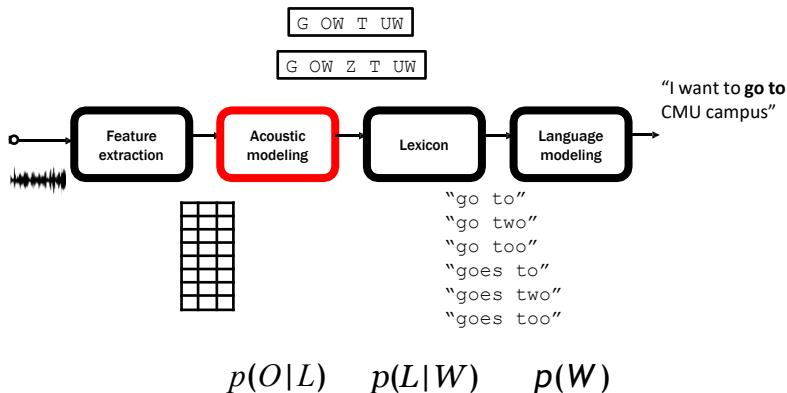
- Performed by so-called **feature extraction** module
 - Mel-frequency cepstral coefficient (MFCC), Perceptual Linear Prediction (PLP) used for **Gaussian mixture model (GMM)**
 - Log Mel filterbank used for **deep neural network (DNN)**
- Time scale
 - 0.0625 milliseconds (16kHz) to 10 milliseconds
- Type of values
 - Scalar (or discrete) to 12-dimensional vector
- **Mostly language-independent process**
 - Some languages use special features, e.g., pitch in Mandarin

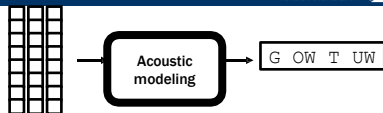
What kind of representation are desired?

- Need to preserve phonetic/linguistic information
- While suppressing irrelevant information (speakers and noises)
- Better to consider the compatibility with backend modules
- Perceptual Linear Prediction (PLP) or multilayer perceptron tandem (MLP-Tandem)
- Learnable frontend (CNN)
- Self-supervised learning (HuBERT)



Speech recognition pipeline





- Performed by so-called **acoustic modeling** module
 - Hidden Markov model (HMM) with **GMM** as an emission probability function
 - Hidden Markov model (HMM) with **DNN** as an emission probability function
- Time scale
 - 10 milliseconds to ~100 milliseconds (depending on a phoneme)
- Type of values
 - 12-dimensional continuous vector to 50 categorical value (~6bit)
- **Mostly language independent**
 - Map of the speech feature (language independent) to phoneme
- It can be a probability of possible phoneme sequences, e.g.,

G	OW	T	UW
---	----	---	----

 or

G	OW	Z	T	UW
---	----	---	---	----

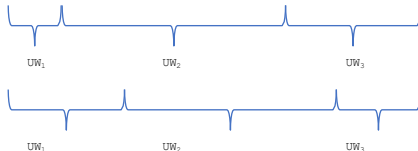
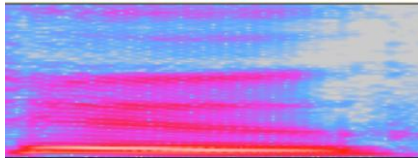
 with some scores

Acoustic model $p(O|L)$

- O and L are **different lengths**
- **Align** speech features and phoneme sequences by using HMM



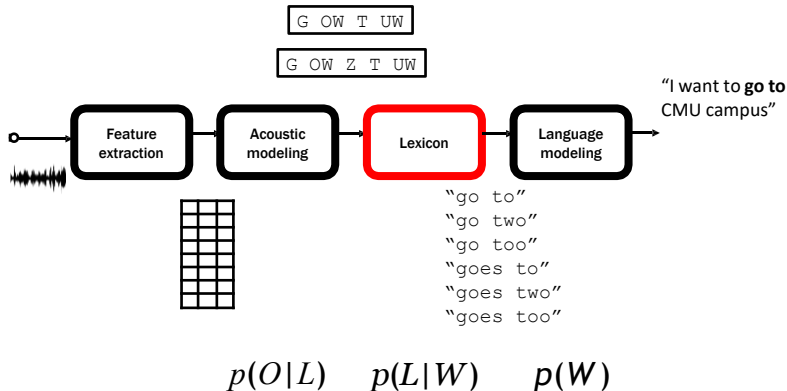
- Provide $p(O|L)$ based on this alignment and model
- The most important problem in speech recognition



or

????

Speech recognition pipeline





- Performed by **lexicon** module
 - American English: CMU dictionary
- Time scale
 - 100 milliseconds (depending on a phoneme) to 1 second (depending on a word and also language)
- Type of values
 - 50 categorical value (~6bit) to 100K categorical value (~2Byte)
- **Language dependent**
- It can be **multiple** word sequences (one to many)

- Basically use a pronunciation dictionary, and map a word to the corresponding phoneme sequence
 - with the probability = 1.0 when single pronunciation
 - with the probability = $1.0/J$ when multiple (J) pronunciations

$$p(L|W) = p(/T/, /OW/ | "two") = 1.0$$

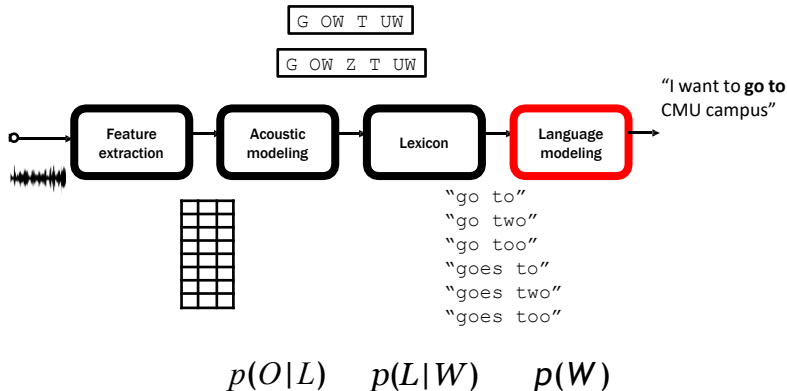
- CMU dictionary
 - <http://www.speech.cs.cmu.edu/cgi-bin/cmudict>

"I want to go to the CMU campus"

→ AY W AA N T T UW G OW T UW DH AH S IY EH M Y UW K AE M P AH S

- Powerful, but limited
- Out of vocabulary issue, especially new word
 - Grapheme2Phoneme mapping based on machine learning

Speech recognition pipeline



- Performed by **language modeling module** $p(W)$
 - N-gram
 - Recurrent neural network language model (RNNLM)
- From training data, we can basically find how possibly "to", "two", and "too" will be appeared after "go"
 - Part of WSJ training data, 37,416 utterances
 - "go to": 51 (times)
 - "go two":
 - "go too":



THE WALL STREET JOURNAL.

- Performed by **language modeling module** $p(W)$
 - N-gram
 - Recurrent neural network language model (RNNLM)
- From training data, we can basically find how possibly "to", "two", and "too" will be appeared after "go"
 - Part of WSJ training data, 37,416 utterances
 - "go to": 51 times
 - "go two": 0 times
 - "go too": 0 times



- Performed by **language modeling module** $p(W)$
 - N-gram
 - Recurrent neural network language model (RNNLM)
- From training data, we can basically find how possibly "to", "two", and "too" will be appeared after "go"
 - WSJ all text data, 6,375,622 sentences
 - "go to": **2710** times
 - "go two":
 - "go too":



- Performed by **language modeling module** $p(W)$
 - N-gram
 - Recurrent neural network language model (RNNLM)
- From training data, we can basically find how possibly "to", "two", and "too" will be appeared after "go"
 - WSJ all text data, 6,375,622 sentences
 - "go to": **2710** (mes
 - "go two": **2** (mes, e.g., "those serving shore plants often go two hundred miles or more"
 - "go too":



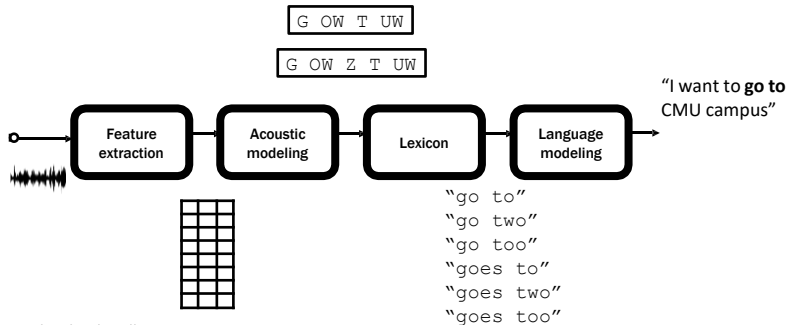
- Performed by **language modeling module** $p(W)$
 - N-gram
 - Recurrent neural network language model (RNNLM)
- From training data, we can basically find how possibly "to", "two", and "too" will be appeared after "go"
 - WSJ all text data, 6,375,622 sentences
 - "go to": **2710** times
 - "go two": **2** times, e.g., "those serving shore plants often go two hundred miles or more"
 - "go too": **41** times, e.g., "he could go too far"



- Performed by **language modeling module** $p(W)$
 - N-gram
 - Recurrent neural network language model (RNNLM)
- From training data, we can basically find how possibly "to", "two", and "too" will be appeared after "go"
 - WSJ all text data, 6,375,622 sentences
 - "go to": **2710** (mes
 - "go two": **2** (mes, e.g., "those serving shore plants o7en go two hundred miles or more"
 - "go too": **41** (times, e.g., "he could go too far"
- **Language dependent**

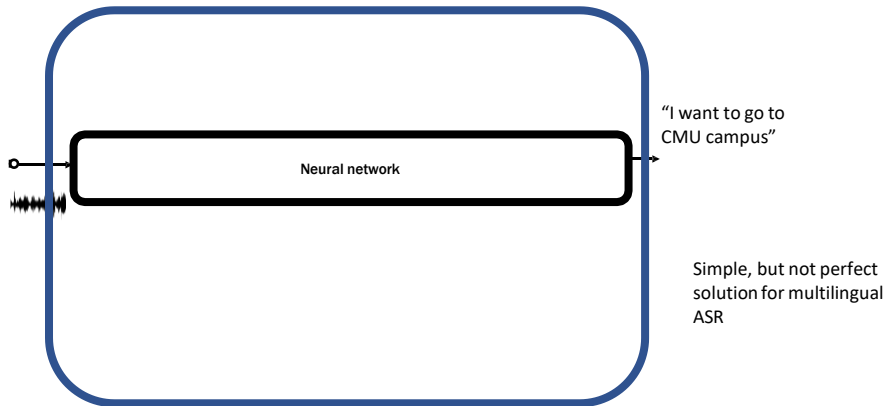


Building speech recognition is really difficult...

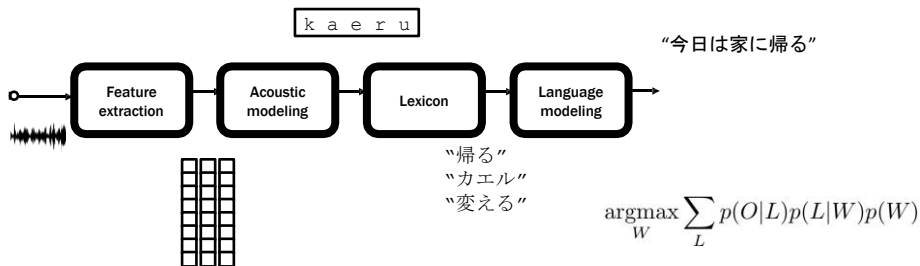


- We need to develop all components
- Each component requires a lot of background knowledge
- We need to tune hyper-parameters in each module

End-to-end speech recognition



How to apply it to the other language?



- We can just change the lexicon and language models (dic\onary and text only data)
- Not easy for end-to-end ASR (we need parallel data)

Transformer & Conformer in ASR

- ▶ **Transformers** and **Conformers** are state-of-the-art architectures for Automatic Speech Recognition (ASR).
- ▶ Both leverage self-attention to model long-range dependencies in audio sequences.
- ▶ Conformer extends Transformer by integrating convolutional modules for local feature extraction.
- ▶ Key components include:
 - Multi-head self-attention
 - Feed-forward networks
 - Residual connections and layer normalization

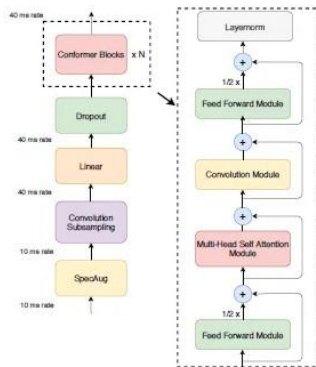


Figure 1: **Conformer encoder model architecture.** Conformer comprises of two macaron-like feed-forward layers with half-step residual connections sandwiching the multi-headed self-attention and convolution modules. This is followed by a post layernorm.

► Multi-Head Self-Attention:

- Captures global context by attending to all positions in the input sequence.
- Enables parallel processing of sequence data.

► Feed-Forward Network (FFN):

- Applies non-linearity and projection to each position independently.
- Typical form:

$$\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2$$

► Residual Connections & Layer Normalization:

- Stabilize training and improve gradient flow.

- ▶ **Conformer** combines self-attention and convolution for effective speech modeling.
- ▶ **Block Structure:**
 - Feed-Forward → Multi-Head Self-Attention + Convolution → Feed-Forward
- ▶ **Key Features:**
 - **Convolution Module:**
 - ▶ Gated Linear Units (GLU)
 - ▶ Depthwise convolution for efficient local feature extraction
 - ▶ Batch normalization for stable training
 - Residual connections and layer normalization throughout

Self-Supervised Learning: Wav2Vec 2.0

wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations

Alexei Baevski

Henry Zhou

Abdelrahman Mohamed

Michael Auli

{[abaeovski](mailto:abaeovski@fb.com),[henryzhou7](mailto:henryzhou7@fb.com),[abdo](mailto:abdo@fb.com),[michaellauli](mailto:michaellauli@fb.com)}@fb.com

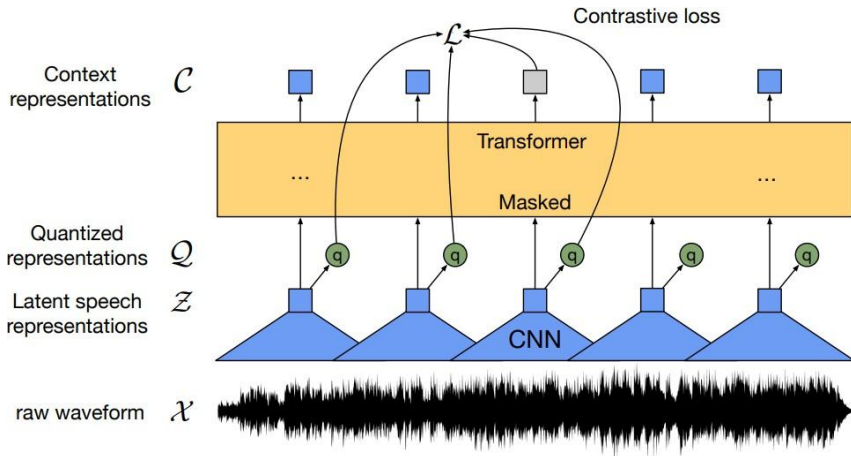
Facebook AI

Abstract

We show for the first time that learning powerful representations from speech audio alone followed by fine-tuning on transcribed speech can outperform the best semi-supervised methods while being conceptually simpler. wav2vec 2.0 masks the speech input in the latent space and solves a contrastive task defined over a quantization of the latent representations which are jointly learned. Experiments using all labeled data of Librispeech achieve 1.8/3.3 WER on the clean/other test sets. When lowering the amount of labeled data to one hour, wav2vec 2.0 outperforms the previous state of the art on the 100 hour subset while using 100 times less labeled data. Using just ten minutes of labeled data and pre-training on 53k hours of unlabeled data still achieves 4.8/8.2 WER. This demonstrates the feasibility of speech recognition with limited amounts of labeled data.¹

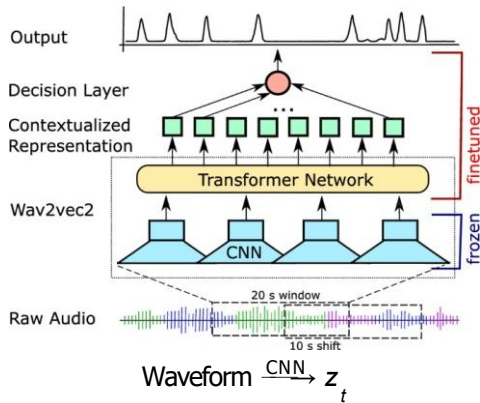
- ▶ **Proposed by Baevski et al. (2020)**
- ▶ **Key Idea:** Self-supervised learning for speech representation.
- ▶ **Architecture:**
 - CNN encoder for raw waveform $\rightarrow$ latent representations z_t
 - Transformer for context representations c_t
 - Quantization module for discrete representations q_t

Wav2Vec 2.0: Overview (cont.)



Wav2Vec 2.0 architecture overview.

- **Step 1:** Raw audio waveform is input to a multi-layer CNN encoder.
- **Output:** Latent speech representations z_t .



- ▶ **Step 2:** Random spans of z_t are masked.
- ▶ **Step 3:** Transformer network produces contextualized representations c_t .

$$z_t \xrightarrow{\text{mask spans} \rightarrow \text{Transformer}} c_t$$

- ▶ **Step 4:** Quantize z_t to discrete representations q_t .
- ▶ **Quantization:** Product of G codebooks, each of size V .

$$z_t \xrightarrow{\text{Quantization}} q_t$$

- **Contrastive Loss:** Distinguish true quantized vector q_t^+ from negatives q^- .

$$L_t = -\log \frac{\exp(\text{sim}(c_t, q_t^+))}{\sum_{q^-} \exp(\text{sim}(c_t, q^-))}$$

- **Diversity Loss:** Encourages usage of all codebook entries.

► State-of-the-art results:

- ~1.8% WER with 960h labeled data
- ~4.8% WER with only 10 minutes labeled data

► **Impact:** Enables efficient use of unlabeled speech data.

TIMIT phoneme recognition accuracy in terms of phoneme error rate (PER).

	dev PER	test PER
CNN + TD-filterbanks [59]	15.6	18.0
PASE+ [47]	-	17.2
Li-GRU + fMLLR [46]	-	14.9
wav2vec [49]	12.9	14.7
vq-wav2vec [5]	9.6	11.6
This work (no LM)		
LARGE (LS-960)	7.4	8.3

Fine-tuning for ASR with Wav2Vec 2.0

- ▶ **Self-Supervised Learning (SSL):** Pre-train Wav2Vec 2.0 on large unlabeled audio.
- ▶ **Fine-tuning:** Use small labeled dataset for ASR.
- ▶ **Loss Function:** Connectionist Temporal Classification (CTC) Loss.
- ▶ **Goal:** Map audio features to text transcriptions.

- ▶ **Small labeled data:** Only 10 minutes of transcribed speech.
- ▶ **Performance:**
 - Word Error Rate (WER): 4.8% (clean) / 8.2% (other)
- ▶ **Efficient:** High accuracy with minimal supervision.

Model	Unlabeled data	LM	dev		test	
			clean	other	clean	other
10 min labeled						
Discrete BERT [4]	LS-960	4-gram	15.7	24.1	16.3	25.2
BASE	LS-960	4-gram	8.9	15.7	9.1	15.6
		Transf.	6.6	13.2	6.9	12.9
LARGE	LS-960	Transf.	6.6	10.6	6.8	10.8
	LV-60k	Transf.	4.6	7.9	4.8	8.2
1h labeled						
Discrete BERT [4]	LS-960	4-gram	8.5	16.4	9.0	17.6
BASE	LS-960	4-gram	5.0	10.8	5.5	11.3
		Transf.	3.8	9.0	4.0	9.3
LARGE	LS-960	Transf.	3.8	7.1	3.9	7.6
	LV-60k	Transf.	2.9	5.4	2.9	5.8
10h labeled						
Discrete BERT [4]	LS-960	4-gram	5.3	13.2	5.9	14.1
Iter. pseudo-labeling [58]	LS-960	4-gram+Transf.	23.51	25.48	24.37	26.02
	LV-60k	4-gram+Transf.	17.00	19.34	18.03	19.92
BASE	LS-960	4-gram	3.8	9.1	4.3	9.5
		Transf.	2.9	7.4	3.2	7.8
LARGE	LS-960	Transf.	2.9	5.7	3.2	6.1
	LV-60k	Transf.	2.4	4.8	2.6	4.9

Wav2Vec 2.0 + CTC Loss (PyTorch)

```
import torch
import torchaudio
from transformers import Wav2Vec2ForCTC, Wav2Vec2Processor

processor = Wav2Vec2Processor.from_pretrained("facebook/
wav2vec2-base-960h")
model = Wav2Vec2ForCTC.from_pretrained("facebook/wav2vec2-
base-960h")

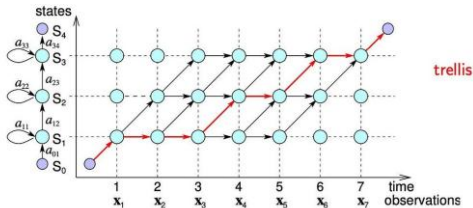
input_audio, _ = torchaudio.load("audio.wav")
inputs = processor(input_audio.squeeze(), sampling_rate
=16000, return_tensors="pt")
with torch.no_grad():
    logits = model(**inputs).logits
pred_ids = torch.argmax(logits, dim=-1)
transcription = processor.decode(pred_ids[0])
print(transcription)
```

► CTC Decoding:

- Greedy decoding: Select most probable token at each timestep.
- Beam search: Explore multiple hypotheses for better accuracy.
- Viterbi algorithm: Find most likely sequence.

► Equation:

$$\hat{y} = \arg \max_y P(y|x)$$



$$p(\mathbf{X}, \text{path}_\ell | \lambda) = p(\mathbf{X} | \text{path}_\ell, \lambda) P(\text{path}_\ell | \lambda)$$

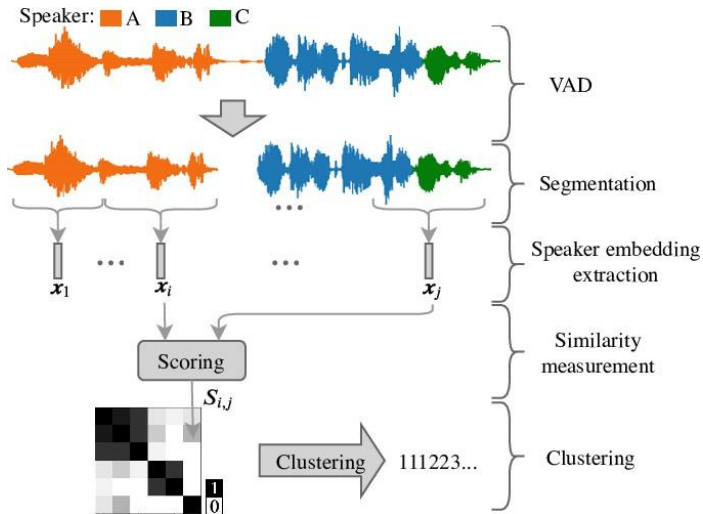
$$\text{likelihood: } \sum_{\{\text{path}_\ell\}} p(\mathbf{X}, \text{path}_\ell | \lambda)$$

$$\text{decode: } \max_{\text{path}_\ell} p(\mathbf{X}, \text{path}_\ell | \lambda)$$

Advanced Tasks: Diarization & Emotion Recognition

- ▶ **Goal:** Identify “who spoke when” in audio streams
- ▶ **Applications:**
 - Meeting transcription
 - Podcast analysis
 - Call center monitoring
- ▶ **Pipeline:**
 1. **Segmentation:** Detect speaker change points
 2. **Embedding:** Extract speaker features
 3. **Clustering:** Group segments by speaker

Speaker Diarization (cont.)

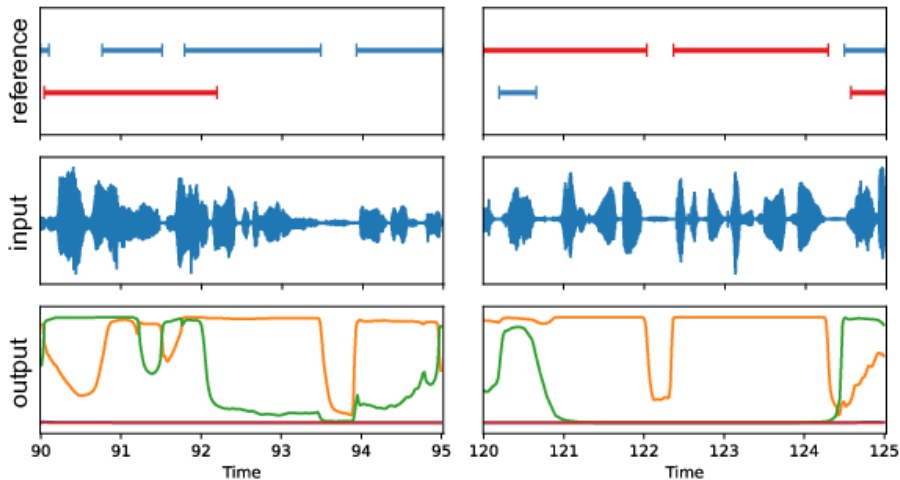


► **Popular Tool:** pyannote-audio

► **Features:**

- Pre-trained diarization models
- Easy integration with Python
- Supports custom pipelines

Speaker Diarization Tools (cont.)



- ▶ **Goal:** Detect emotions from speech signals
- ▶ **Applications:**
 - Human-computer interaction
 - Mental health monitoring
 - Customer service analytics
- ▶ **Datasets:** IEMOCAP, YouTube, arXiv, PLOS

Voice Signal → Feature Extraction → Model Training → Emotions



Min Max Range

Mean Variance



Time Domain

Frequency Domain



Support Vector Machine
k-nearest neighbor
Naïve Bayes
Decision Tree
Gaussian Mixture



Neural Networks



Angry

Happy



Sad

Neutral



Fear

► Wav2Vec 2.0:

- Self-supervised speech representation
- Robust to noise and speaker variation

► Neural CDEs:

- Continuous-time neural networks
- Capture temporal dynamics in speech

► Performance:

- Achieves ~70% accuracy on IEMOCAP

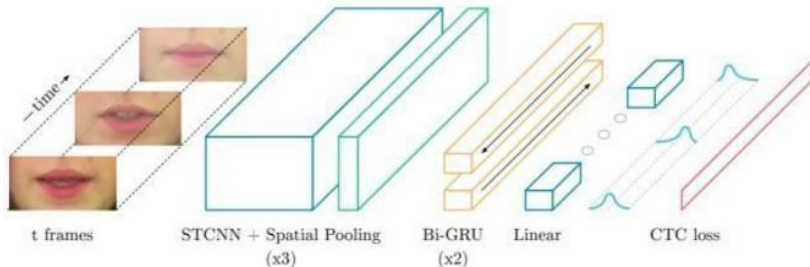
$$\text{Accuracy} = \frac{\text{Correct Predictions}}{\text{Total Samples}}$$

► LipNet:

- Integrates lip movement and audio
- Robust speech decoding in noisy environments

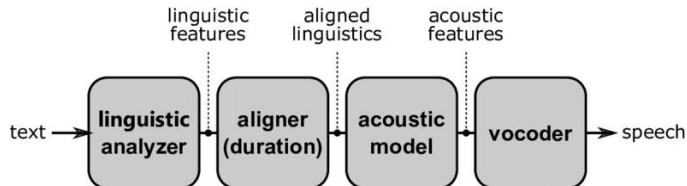
► Benefits:

- Improved accuracy
- Handles missing or corrupted audio



Text-to-Speech (TTS) & Vocoders

- ▶ Converts text input into human-like speech
- ▶ Applications: virtual assistants, accessibility, entertainment
- ▶ Key components:
 - Text analysis
 - Acoustic modeling
 - Vocoder synthesis

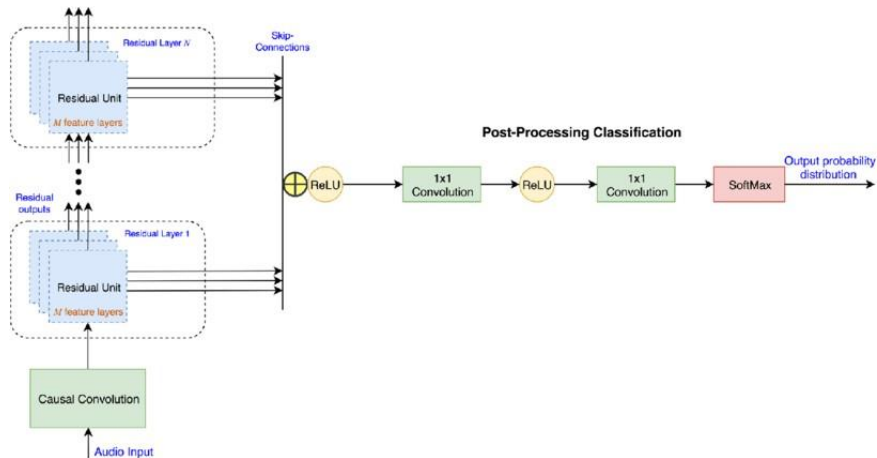


- ▶ Deep generative model for raw audio
- ▶ Autoregressive: predicts next sample given previous samples
- ▶ High-quality, natural-sounding speech
- ▶ Equation:

$$p(x) = \prod_{t=1}^T p(x_t | x_1, \dots, x_{t-1})$$

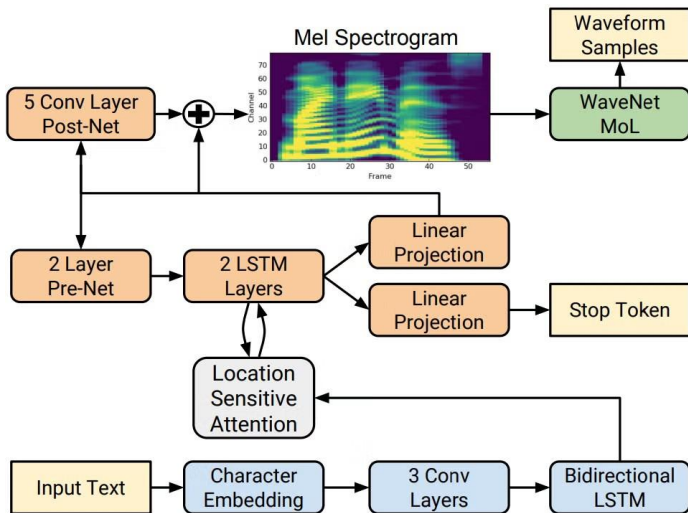
- ▶ Computationally expensive for real-time use

WaveNet (2016) (cont.)



- ▶ End-to-end TTS system
- ▶ Pipeline:
 - Text $\rightarrow$ Mel-spectrogram (sequence-to-sequence model)
 - Mel-spectrogram $\rightarrow$ WaveNet/GAN vocoder (audio synthesis)
- ▶ Improved prosody and pronunciation
- ▶ Flexible for different voices and languages

Tacotron 2 (cont.)



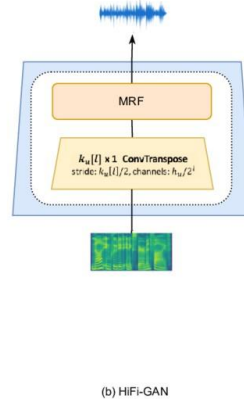
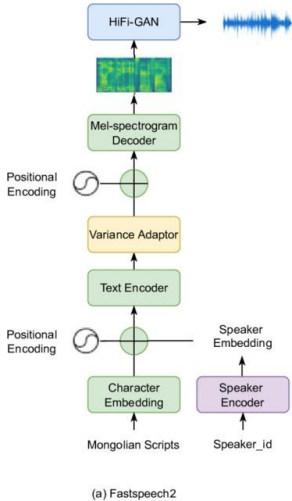
- ▶ Non-autoregressive models: faster inference
- ▶ FastSpeech:
 - Parallel generation of spectrograms
 - Duration prediction for alignment
- ▶ HiFi-GAN:
 - GAN-based vocoder
 - High-fidelity, real-time synthesis

► Glow-TTS:

- Flow-based generative model
- Efficient and expressive speech synthesis

► Diffusion models:

- Iterative refinement of audio
- State-of-the-art naturalness



Challenges & Current Research

- ▶ **Noise robustness and domain mismatch**

Models often struggle with noisy environments and data from unseen domains.

- ▶ **SSL models need massive compute**

Self-supervised learning approaches require significant computational resources.

- ▶ **Interpretability & data biases**

Understanding model decisions and mitigating biases in datasets remain open problems.

- ▶ **Cross-modal fusion is still hard**

Integrating audio with other modalities (e.g., text, vision) is challenging.

- ▶ Robust audio processing demands both signal-level and deep representation learning.
- ▶ ASR now thrives on SSL + Transformer architectures.
- ▶ Conformer combines locality and global context.
- ▶ Advanced tasks highlight the breadth of audio-NLP.
- ▶ Emerging avenues: TTS, diffusion, multimodal AI, efficient edge solutions.

References

- 1 Baevski, A., Zhou, Y., Mohamed, A., & Auli, M. (2020). wav2vec 2.0: A framework for self-supervised learning of speech representations. *arXiv preprint arXiv:2006.11477*.
- 2 Gulati, A., et al. (2020). Conformer: Convolution-augmented Transformer for Speech Recognition. *arXiv preprint arXiv:2005.08100*.
- 3 Van den Oord, A., Dieleman, S., Zen, H., et al. (2016). WaveNet: A generative model for raw audio. *arXiv preprint arXiv:1609.03499*.
- 4 Kim, Y., Kong, J., & Son, J. (2020). FastSpeech: Fast, Robust and Controllable Text to Speech. *arXiv preprint arXiv:2006.04558*.
- 5 Lopez-Moreno, I., et al. (2021). DiffWave: A Versatile Diffusion Model for Audio Synthesis. *arXiv preprint arXiv:2009.09761*.
- 6 SpeechBrain Documentation. <https://speechbrain.readthedocs.io>
- 7 torchaudio Documentation. <https://pytorch.org/audio/stable/index.html>

- 8 NVIDIA NeMo Documentation.
<https://docs.nvidia.com/deeplearning/nemo/user-guide/docs/en/main/>
- 9 NVIDIA Riva Documentation. <https://docs.nvidia.com/riva/>
- 10 Tripathi, S., et al. (2022). Emotion Recognition from Speech using Wav2Vec2 and CDE. *arXiv preprint arXiv:2209.12345*.
- 11 Whisper: Multilingual Speech Recognition.
<https://github.com/openai/whisper>
- 12 Jont Allen's Speech Page. <http://jontalle.web.engr.illinois.edu>